

verbos_http

- **VERBOS OU MÉTODOS HTTP**
 - Os verbos HTTP, também conhecidos como métodos HTTP, são comandos que indicam a ação desejada a ser realizada em um recurso na web. Aqui estão os principais verbos HTTP
 - O que é idempotente?
 - Qual termo contrário a idempotente?
 - No contexto do send request do cliente existe diferença?
 - Existe alguma técnica para saber se o post foi bem sucedido?
 - O código de retorno 200 nas ações POST não é seguro?

VERBOS OU MÉTODOS HTTP

Os verbos HTTP, também conhecidos como métodos HTTP, são comandos que indicam a ação desejada a ser realizada em um recurso na web. Aqui estão os principais verbos HTTP

1. **GET**: Solicita a recuperação de um recurso. Não deve alterar o estado do servidor, sendo considerado um método seguro e idempotente.
2. **POST**: Envia dados ao servidor para criar ou modificar um recurso. É utilizado principalmente para enviar formulários ou dados de uma aplicação para o servidor.
3. **PUT**: Atualiza ou cria um recurso em uma URL específica. É idempotente, o que significa que várias requisições PUT com os mesmos dados não devem alterar o estado do recurso além da primeira requisição.
4. **DELETE**: Remove um recurso identificado pela URL. Como PUT, é idempotente.
5. **PATCH**: Aplica modificações parciais a um recurso. Diferente do PUT, que substitui o recurso completo, o PATCH altera apenas os campos fornecidos.
6. **HEAD**: Solicita as mesmas informações que o GET, mas sem o corpo da resposta. É utilizado para obter metadados, como cabeçalhos HTTP.
7. **OPTIONS**: Retorna os métodos HTTP suportados por um recurso. Útil para verificar quais operações são permitidas em um endpoint.
8. **TRACE**: Realiza um loopback de uma mensagem de solicitação para fins de teste e diagnóstico, retornando a requisição ao cliente.
9. **CONNECT**: Estabelece um túnel para o servidor identificado pela URL de destino. É usado principalmente para conectar o cliente a um servidor proxy, especialmente para comunicações seguras via HTTPS.
10. **Compatibilidade dos browsers**

O que é idempotente?

1. Idempotente é um termo usado em computação para descrever uma operação que pode ser aplicada várias vezes sem alterar o resultado além da primeira aplicação. Em outras palavras, se uma operação é idempotente, o efeito de executá-la uma vez ou várias vezes com os mesmos parâmetros é o mesmo.
2. No contexto dos verbos HTTP:
 1. *GET*: É idempotente porque solicitar o mesmo recurso várias vezes não altera o estado do servidor. Você sempre recebe a mesma resposta (assumindo que o recurso não mudou entre as requisições).
 2. *PUT*: Também é idempotente. Submeter uma mesma requisição *PUT* várias vezes com os mesmos dados resultará no mesmo estado do recurso no servidor, independentemente de quantas vezes a operação for repetida.
 3. *DELETE*: É idempotente porque a exclusão de um recurso já inexistente não terá efeito adicional; após a primeira exclusão, requisições subsequentes de *DELETE* não terão nenhum impacto.
 4. *POST*: Não é idempotente, pois cada requisição pode criar um novo recurso ou alterar o estado do servidor de maneira diferente. Por exemplo, enviar várias requisições *POST* para criar um novo recurso pode resultar em múltiplas criações.
3. Em resumo, operações idempotentes garantem que múltiplas execuções da mesma ação não vão gerar efeitos colaterais adicionais após a primeira execução.

Qual termo contrário a idempotente?

1. O termo oposto a idempotente é *não idempotente*. Em computação, uma operação não idempotente é aquela em que a repetição da operação com os mesmos parâmetros pode causar efeitos diferentes a cada vez que é realizada.
2. No contexto dos verbos HTTP:
 1. *POST* é o exemplo mais comum de uma operação *não idempotente*. Por exemplo, enviar a mesma requisição *POST* várias vezes pode criar vários novos recursos, cada um diferente, resultando em um estado diferente do servidor a cada repetição.
3. Operações *não idempotentes* podem causar efeitos cumulativos ou mudanças de estado que diferem a cada execução, ao contrário das operações idempotentes, que mantêm o mesmo resultado após a primeira execução.

No contexto do send request do cliente existe diferença?

1. Sim, no contexto de um cliente HTTP realizando o envio de requisições (por exemplo, utilizando `TMi_FpHttpClient` ou outro cliente HTTP no Free Pascal), a diferença entre métodos idempotentes e não idempotentes é importante, especialmente em termos de comportamento em caso de falhas de rede ou necessidade de repetição da requisição.

2. Idempotente (e.g., GET, PUT, DELETE)

1. Repetição Segura: Se uma requisição falhar ou houver incerteza se ela foi recebida pelo servidor, o cliente pode reenviar a requisição sem se preocupar em causar efeitos colaterais adicionais. Por exemplo, se você fizer um GET e a conexão for interrompida, pode simplesmente tentar novamente.
2. Uso em retentativas: O cliente HTTP pode implementar estratégias automáticas de repetição (retry) para esses métodos em caso de falha, porque a operação não vai alterar o estado do servidor de maneira inesperada.

3. Não Idempotente (e.g., POST)

1. Cuidado com Repetições: Se uma requisição POST falhar ou se a resposta não for recebida pelo cliente, há um risco ao repetir a requisição, pois isso pode resultar em múltiplas criações ou alterações inesperadas no servidor. Por exemplo, repostar um formulário de compra pode resultar em múltiplas transações.
2. Gerenciamento de Requisições: Para métodos não idempotentes, os clientes geralmente precisam de mecanismos adicionais para garantir que a requisição foi bem-sucedida antes de tentar novamente (como tokens de transação ou verificações adicionais).

4. No Free Pascal

1. Quando você usa SendRequest ou funções equivalentes no TMI_FpHttpClient, entender se o método HTTP que você está usando é idempotente ou não ajuda a decidir como lidar com erros ou interrupções de rede. Por exemplo, você pode ser mais agressivo em repetir requisições GET, mas deve ser muito mais cauteloso com POST.
2. Essa distinção é crucial em aplicações que precisam garantir consistência e integridade dos dados, especialmente em cenários de alta disponibilidade e resiliência.

Existe alguma técnica para saber se o post foi bem sucedido?

1. Sim, há várias técnicas que podem ser usadas para garantir ou verificar se uma requisição POST foi bem-sucedida, especialmente em cenários onde a repetição da requisição pode causar problemas (como a criação de múltiplos recursos indesejados). Aqui estão algumas estratégias comuns:

1. Respostas HTTP Status

1. Status Codes: Um servidor geralmente responde com um código de status HTTP indicando o resultado da requisição. Para um POST, o código 200 OK ou 201 Created geralmente indica sucesso.
2. Verifique o Conteúdo da Resposta: Além do código de status, o conteúdo da resposta pode incluir informações importantes, como um identificador do recurso criado. Validar essa informação pode confirmar o sucesso.

2. Utilização de Tokens de Idempotência

1. Token de Idempotência: Enviar um token único junto com cada requisição POST. O servidor armazena esse token e garante que, se o mesmo token for recebido novamente, a requisição duplicada não será processada, retornando a mesma resposta da primeira tentativa. Isso ajuda a evitar duplicatas em casos onde o cliente não tem certeza se a requisição foi bem-sucedida.

1. Exemplo: Cabeçalho Idempotency-Key é um padrão em algumas APIs RESTful.

2. Confirmar Recebimento

1. Recibo de Confirmação: O servidor pode enviar um recibo com um identificador único após a criação do recurso, como um ID de transação ou número de pedido. O cliente pode então armazenar esse identificador e, em caso de falha de rede, verificar com o servidor se esse ID já foi processado.
2. Verificação de Estado: Após enviar um POST, o cliente pode fazer uma requisição GET para confirmar que o recurso foi criado ou o estado foi alterado conforme esperado.

3. Transações Atômicas no Servidor

1. Processamento Atômico: O servidor pode garantir que a operação é atômica, o que significa que a operação ou será completada totalmente ou não será realizada de forma alguma. Isso é especialmente útil para operações complexas que envolvem múltiplos passos.

1. Retry com Backoff Exponencial

1. Retry com Backoff: Se uma requisição POST falhar por um erro temporário, a técnica de retry com backoff exponencial pode ser usada para tentar novamente após um intervalo crescente de tempo, reduzindo a carga no servidor e dando mais tempo para a recuperação. No entanto, essa técnica deve ser usada com cautela para POST devido ao risco de duplicação.

2. Logs de Auditoria

1. Logs de Auditoria: Manter um registro detalhado das operações POST e suas respostas pode ajudar a identificar e resolver problemas quando há dúvidas se a requisição foi bem-sucedida.

3. Persistência Local e Confirmação

1. Cache Local Temporário: O cliente pode salvar a requisição localmente até receber uma confirmação do servidor. Se não houver confirmação, o cliente pode reprocessar ou reavaliar a necessidade de reenviar a requisição.

4. Implementação no Free Pascal

1. No Free Pascal, ao usar `TMi_FpHttpClient`, você pode verificar o código de status da resposta (`ResponseStatusCode`) e analisar o conteúdo da resposta (`Response.Content`). Se o servidor suporta tokens de idempotência, você pode adicionar um cabeçalho personalizado na requisição usando `AddHeader`.
2. Essas técnicas, quando implementadas adequadamente, ajudam a garantir que sua aplicação lide corretamente com a incerteza na comunicação HTTP, minimizando os efeitos indesejados de requisições duplicadas ou falhas de rede.

O código de retorno 200 nas ações POST não é seguro?

1. O código de status HTTP 200 OK indica que a requisição foi bem-sucedida e que o servidor processou a solicitação com sucesso. No entanto, quando usado em operações POST, o código 200 OK pode não ser o mais adequado ou seguro para todas as situações, especialmente quando o POST está criando novos recursos ou realizando ações que alteram o estado do servidor.

1. Por que o 200 OK pode não ser ideal em POST?

1. Ambiguidade: O código 200 OK é genérico e pode ser usado para uma ampla gama de situações. Isso pode tornar difícil para o cliente discernir se a requisição POST realmente realizou a ação esperada, especialmente quando se espera que o POST crie um novo recurso.
2. Criação de Recursos: Quando um POST é usado para criar um novo recurso, o código mais apropriado é o 201 Created. Esse código indica claramente que um novo recurso foi criado no servidor e geralmente inclui um cabeçalho `Location` que aponta para o URI do novo recurso. Isso oferece uma confirmação mais específica e confiável para o cliente.
3. Verificação de Sucesso Completo: Em alguns casos, o 200 OK pode ser retornado mesmo que a operação tenha sido parcialmente bem-sucedida, ou seja, a ação foi realizada, mas talvez com algumas advertências ou problemas. Isso pode não ser suficiente para operações críticas onde a confirmação completa da ação é necessária.

2. Códigos de Status Mais Apropriados para POST

1. 201 Created: Indica que a requisição POST foi bem-sucedida e que um novo recurso foi criado. É o código de status ideal quando o POST resulta na criação de um recurso no servidor.
2. 202 Accepted: Indica que a requisição foi aceita para processamento, mas ainda não foi concluída. Esse código pode ser usado quando o processamento é assíncrono, e o cliente pode precisar consultar o progresso ou o resultado posteriormente.
3. 204 No Content: Indica que a requisição foi processada com sucesso, mas não há conteúdo adicional a ser retornado no corpo da resposta. Útil para casos onde a operação foi concluída, mas não há dados novos ou um recurso criado.

4. 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, etc.: Esses códigos indicam que houve um problema com a requisição, e o cliente precisa corrigir ou autenticar-se antes de tentar novamente.

3. Práticas Recomendadas

1. Verificação de Conteúdo da Resposta: Além de checar o código de status, verifique o conteúdo da resposta para confirmar se a operação foi realizada conforme esperado. O servidor pode retornar um JSON ou outro formato que inclua detalhes sobre a operação.
2. Cabeçalhos e Identificadores: Se for recebido um código 200 OK, a resposta deve, idealmente, incluir detalhes suficientes (como identificadores de recursos ou confirmações explícitas) para o cliente determinar se a operação foi bem-sucedida e o que exatamente foi alterado ou criado.
3. Uso de 201 Created para Criação: Sempre que o POST criar um novo recurso, o servidor deve preferencialmente retornar 201 Created para maior clareza e segurança.

4. Implementação no Free Pascal

1. Ao usar TMi_FpHttpClient ou outro cliente HTTP no Free Pascal, é possível verificar o ResponseStatusCode e tomar decisões baseadas no código retornado. A análise do conteúdo da resposta (Response.Content) também é fundamental para confirmar se a operação POST foi realizada com sucesso.
2. Em resumo, enquanto o 200 OK é um código de sucesso, ele pode não ser a escolha mais informativa ou segura para operações POST, especialmente em casos de criação de recursos. Códigos como 201 Created oferecem uma confirmação mais explícita e segura de que a operação foi realizada conforme esperado.

